KONTROL

ESTÁNDAR MAESTRO DE FRONTEND Y ARQUITECTURA TÉCNICA

Reglas obligatorias de diseño, interacción, componentes, validación, accesibilidad, pruebas, reportes y despliegue

Documento preventivo para construir KONTROL desde cero sin repetir el retrabajo realizado en el frontend de INNOVA

Campo	Definición
Producto	KONTROL - Plataforma Administrativa Empresarial
Versión	1.0
Fecha	3 de agosto de 2026
Ámbito	Frontend, backend de soporte, reportes, infraestructura y QA
Aplicación	Obligatoria para todas las Specs, historias, tareas, componentes y rutas
Clasificación	Confidencial - Documento normativo interno del proyecto

Decisión de calidad

Este documento es una puerta de calidad. Ninguna pantalla, componente o flujo de KONTROL puede aprobarse si contradice estas reglas o si no posee una excepción documentada mediante ADR y aprobada por Producto, Arquitectura, UI/UX y QA.

Control del documento

Elemento	Definición
Propósito	Fijar de antemano todas las reglas del frontend y del stack técnico para reducir defectos, inconsistencias y retrabajo.
Base de experiencia	Lecciones derivadas de la auditoría y modernización del frontend de INNOVA, especialmente modales, tablas, validaciones, loaders, notificaciones, permisos, responsive y consistencia global.
Documento de referencia	Manual Técnico de INNOVA v3.0.0, usado como antecedente de arquitectura y no como diseño a copiar.
Autoridad	La Constitución del proyecto, las Specs aprobadas y este estándar. En caso de conflicto, se debe abrir una clarificación y un ADR.
Mantenimiento	Toda modificación requiere versión, motivo, impacto, aprobación y actualización de casos de prueba.
Alcance	Aplicación web, API, contratos de error, generación de PDF, Docker Compose, pruebas y despliegue por cliente.

Resumen ejecutivo

KONTROL se construirá con un sistema de diseño y una infraestructura de componentes obligatorios antes de desarrollar módulos de negocio. No se permitirá que cada equipo resuelva formularios, tablas, mensajes, loaders, diálogos o permisos de manera distinta. La experiencia debe ser predecible, accesible, responsive, auditable y consistente en todas las rutas.

- No se utilizarán modales para formularios complejos, matrices, DataTables, configuraciones extensas ni procesos que requieran navegación o varios minutos de trabajo.
- Toda acción asíncrona tendrá estado de carga dentro del control que la inició, prevención de doble envío, resultado de éxito o error y trazabilidad.
- Los mensajes de éxito y error permanecerán visibles hasta cierre manual; no desaparecerán antes de que el usuario pueda leerlos.
- Las tablas no tendrán desplazamiento horizontal. La información se priorizará, adaptará o trasladará a detalle expandible/tarjetas.
- Los formularios utilizarán reglas comunes de validación, foco, tipos de datos, moneda, fechas, números y catálogos grandes.
- El backend será la autoridad para permisos, integridad referencial, licencia, estados y acciones permitidas.
- Cada pantalla tendrá URL propia, encabezado, breadcrumbs, estados de carga, vacío, error, prohibido y no encontrado.
- La calidad se verificará con pruebas unitarias, componentes, integración, E2E, accesibilidad y regresión visual.

Contenido

- 1. Principios y decisiones no negociables
- 2. Stack tecnológico aprobado
- 3. Arquitectura de solución y estructura de repositorio
- 4. Sistema de diseño y tokens
- 5. App Shell, navegación y rutas
- 6. Tipos de pantallas y wireframes base
- 7. Política de modales, diálogos, popovers y tooltips
- 8. Botones, acciones asíncronas, loaders y mensajes
- 9. Formularios, campos y validación
- 10. DataTables, búsquedas, filtros y paginación
- 11. Estados de pantalla y manejo de errores
- 12. Crear, editar, activar, desactivar, eliminar y anular
- 13. Responsive y accesibilidad
- 14. Estado, caché, API y concurrencia
- 15. Seguridad, permisos, licencia y auditoría
- 16. Reportes, impresión, exportación y archivos
- 17. Rendimiento y observabilidad

- 18. Pruebas y puertas de calidad
- 19. Docker Compose, despliegue y configuración
- 20. Estructura de componentes obligatorios
- 21. Definition of Done y checklist por pantalla
- 22. Matriz maestra de reglas
- 23. Referencias técnicas

1. Principios y decisiones no negociables

Código	Principio	Regla
P-001	Consistencia global	Una regla visual o de interacción debe resolverse una sola vez en un componente compartido. No se permiten variantes locales sin ADR.
P-002	Pantallas completas	Crear, editar, configurar, revisar permisos, trabajar matrices y ejecutar procesos complejos se hace en rutas completas.
P-003	Backend como autoridad	Permisos, licencia, estado, integridad referencial y acciones permitidas se validan en backend y base de datos.
P-004	Feedback obligatorio	Toda acción comunica carga, éxito o error. El silencio no es un estado válido.
P-005	Sin pérdida de datos	Cambios sin guardar deben detectarse y confirmarse antes de abandonar la ruta.
P-006	Accesibilidad desde origen	Teclado, foco, labels, aria, contraste y lectores de pantalla forman parte de la implementación, no de una fase posterior.
P-007	Responsive real	No se comprime una tabla de escritorio en móvil. Se adapta el patrón de interacción.
P-008	Sin scroll horizontal	Las pantallas y tablas no deben depender de desplazamiento horizontal en resoluciones soportadas.
P-009	Dominio explícito	El tipo de control y texto debe corresponder a la naturaleza real del dato. Códigos y referencias son texto, no números.
P-010	Reglas versionadas	Impuestos, monedas, permisos y reglas de negocio no se hardcodean en componentes.
P-011	Idempotencia	Acciones críticas deben resistir reintentos sin duplicar documentos, movimientos ni pagos.
P-012	Trazabilidad	Toda operación relevante genera un evento auditable y una referencia de solicitud.

Prohibición expresa

No se aprobará una pantalla bajo la justificación de que “funciona” si no cumple el estándar visual, funcional, accesible, responsive y de pruebas. La ausencia de error técnico no equivale a calidad de producto.

2. Stack tecnológico aprobado

Las versiones se fijarán en archivos de bloqueo. Los rangos siguientes definen la línea tecnológica; cualquier cambio mayor requiere ADR, pruebas de compatibilidad y aprobación. Se seleccionan tecnologías con documentación oficial y comunidades maduras.

Capa	Tecnología	Línea	Uso
Frontend	React	19.x	Componentes, rendering y UI declarativa.
Frontend	TypeScript	6.x o estable compatible aprobada	Tipado estricto; no se permite JavaScript nuevo sin excepción.
Build	Vite	8.x	Desarrollo, HMR y build de producción.
Estilos	Tailwind CSS	4.x	Tokens y utilidades controladas; no estilos improvisados por pantalla.
Rutas	React Router	8.x	Rutas completas, loaders, boundaries y navegación.
Servidor state	TanStack Query	5.x	Consultas, caché, mutaciones e invalidación.

Capa	Tecnología	Línea	Uso
Formularios	React Hook Form	7.x	Estado de formularios y rendimiento.
Validación UI	Zod	4.x	Esquemas tipados; alineados con contratos de API.
Tablas	TanStack Table	8.x	Motor headless para tablas estandarizadas.
Iconos	Lucide React	Línea estable	Única librería de iconos de aplicación.
Fechas	date-fns + Intl	Línea estable	Formateo, zonas horarias y operaciones de fecha.
Backend	Python	3.13.x	Lenguaje del backend y generación de reportes.
API	FastAPI	Línea estable compatible	REST/OpenAPI, validación y dependencias.
Modelos	Pydantic	2.x	DTO y validación del backend.
ORM	SQLAlchemy	2.x	Persistencia y transacciones.
Migraciones	Alembic	1.x	Migraciones upgrade/downgrade revisadas.
Base de datos	PostgreSQL	18.x	Datos transaccionales, restricciones, índices y auditoría.
PDF	ReportLab	4.x	Reportes, documentos y salidas PDF controladas.
Plantillas	Jinja2	3.x	Correos y plantillas no fiscales cuando aplique.
Trabajos	Celery + Redis	5.x / 7.x	Reportes pesados, correos y tareas programadas.
Proxy	Nginx	Línea estable	SPA, reverse proxy, headers y compresión.
Contenedores	Docker Engine + Compose	Compose Specification	Instalación independiente por cliente.
Unit frontend	Vitest	4.x	Pruebas unitarias y de componentes.
E2E	Playwright	Línea estable	Chromium, Firefox y WebKit; responsive y visual.
Unit backend	Pytest	8.x	Reglas, servicios, API y base de datos.
Calidad TS	ESLint + oxlint + Prettier	Líneas compatibles	Lint, velocidad y formato.
Calidad Python	Ruff + mypy	Líneas compatibles	Lint, formato y tipado.
Contratos	OpenAPI + cliente generado	OpenAPI 3.1	Evitar DTO duplicados manualmente.

Regla de versiones

No se usan dependencias “latest” en producción. Se fija versión exacta en lockfiles e imágenes, se habilitan actualizaciones controladas y se prueba cada actualización en CI antes de incorporarla.

2.1 Decisiones técnicas específicas

- La aplicación será una SPA empresarial servida por Nginx y conectada al backend por `/api/v1`.
- FastAPI publicará OpenAPI; el cliente TypeScript se generará automáticamente y no se mantendrán interfaces duplicadas a mano.
- PostgreSQL será la única fuente de verdad transaccional. Redis no reemplazará datos permanentes.
- ReportLab generará PDF en backend. El navegador no será responsable de producir documentos fiscales definitivos.
- Las imágenes Docker serán multi-stage, no contendrán secretos y se ejecutarán con usuario no root.
- Cada cliente tendrá instalación independiente mediante Docker Compose, volúmenes persistentes, configuración propia, licencia firmada y respaldo separado.

3. Arquitectura de solución y estructura de repositorio

```
kontrol/
├── kontrol.py                    # CLI de instalación y operación
├── compose.yaml                  # Producción
├── compose.dev.yaml              # Desarrollo
├── .env.example
├── resources/
│   ├── frontend/
│   │   ├── src/app/              # providers, router, shell
│   │   ├── src/modules/          # módulos por dominio
│   │   ├── src/components/       # componentes compartidos
│   │   ├── src/design-system/    # tokens y primitivas
│   │   ├── src/api/              # cliente OpenAPI generado
│   │   ├── src/lib/              # utilidades controladas
│   │   └── src/tests/
```

```
│   └── backend/
│       ├── app/core/              # config, seguridad, DB, licencia
│       ├── app/modules/           # dominios
│       ├── app/shared/            # errores, auditoría, reportes
│       ├── alembic/
│       └── tests/
├── specs/
├── requerimientos/
├── docs/
├── scripts/
├── storage/                       # documentos persistentes
├── backups/
└── databases/                     # volumen PostgreSQL local si aplica
```

Servicio Compose	Responsabilidad	Persistencia
frontend	Build React servido por Nginx; proxy a API.	No requiere datos persistentes.
backend	FastAPI/Uvicorn; reglas, autenticación, permisos, licencia, reportes.	Archivos temporales controlados.
worker	Celery para reportes, correos y trabajos largos.	Cola en Redis; resultados según política.
scheduler	Celery Beat o servicio programador.	Programación declarativa.
db	PostgreSQL 18.	Volumen obligatorio y respaldado.
redis	Cola, locks y caché efímera.	Volumen opcional; no fuente de verdad.
storage	Volumen o MinIO opcional para documentos.	Persistencia obligatoria y respaldada.

3.1 Estructura modular del frontend

- Cada módulo tendrá `routes`, `pages`, `components`, `queries`, `mutations`, `schemas`, `types`, `permissions` y `tests`.
- Los componentes compartidos no pueden importar módulos de negocio. Los módulos sí pueden importar el design system y componentes shared.
- No se permite un archivo `utils.ts` genérico sin responsabilidad clara. Las utilidades se agrupan por dominio técnico.
- Las rutas se cargan de forma diferida por módulo y deben tener error boundary.
- Los textos visibles se centralizan para mantener consistencia y preparar internacionalización.

4. Sistema de diseño y tokens

Antes de construir módulos se debe cerrar un Design System mínimo funcional. No se permitirá que cada pantalla defina colores, radios, sombras, tamaños o espaciados arbitrarios.

Grupo	Tokens obligatorios
Color	brand, surface, background, border, text, muted, success, warning, danger, info, focus, disabled.
Tipografía	Ubuntu: light, regular, medium, bold; escala tipográfica, peso, interlineado y truncamiento.
Espaciado	Escala única de 4 px: 4, 8, 12, 16, 20, 24, 32, 40, 48.
Radios	small, medium, large; uso definido por control, tarjeta y panel.
Sombras	Niveles controlados; no usar sombra como única separación.
Z-index	base, sticky, dropdown, popover, tooltip, drawer, dialog, notification.
Tamaños	Altura de controles, iconos, botones, sidebar, topbar y acciones.
Breakpoints	mobile, tablet, laptop, desktop, wide; documentados y probados.

4.1 Tipografía

- La tipografía institucional de la aplicación será Ubuntu.
- Los títulos de página, secciones, labels, ayudas, mensajes y tablas tendrán estilos definidos; no se asignan tamaños locales.
- Los textos no deben escribirse en mayúsculas completas salvo códigos o encabezados muy breves.
- Los números tabulares en reportes y tablas financieras deben alinearse y usar formato consistente.
- El contraste mínimo será WCAG 2.2 AA.

4.2 Iconografía

- Lucide React será la única librería de iconos de la interfaz.

- Editar: `Pencil`; eliminar: `Trash2`; desactivar: `CircleX` o `X`; activar: `CircleCheck` o `Check`; ver: `Eye`; descargar: `Download`; imprimir: `Printer`.
- Los iconos no sustituyen el nombre accesible. Todo botón icon-only tendrá `aria-label` y tooltip.
- No utilizar iconos distintos para la misma acción en módulos diferentes.
- No utilizar emojis como iconos funcionales.

5. App Shell, navegación y rutas

Elemento	Regla
Sidebar	Colapsable; muestra solo una rama expandida cuando sea necesario; opción activa visible; no cubre contenido.
Topbar	Empresa activa, sucursal/establecimiento, usuario, fecha confiable, licencia y salida.
Contenido	Ancho fluido con máximos por tipo de página; nunca queda debajo del sidebar.
Breadcrumbs	Obligatorios en páginas de segundo nivel o superior.
Título	Un solo H1 por ruta; nombre claro y contexto.
Acciones	Alineadas a la derecha; primaria única; acciones secundarias jerarquizadas.
URL	Cada lista, alta, edición, detalle, permisos y proceso complejo tiene URL propia.
Estado de lista	Búsqueda, página, orden y filtro de estado se conservan al regresar.

5.1 Convenciones de rutas

```
/maestros/clientes
/maestros/clientes/nuevo
/maestros/clientes/:id
/maestros/clientes/:id/editar
/maestros/productos
/maestros/productos/:id/editar
/ventas/facturas/nueva
/ventas/facturas/:id
/seguridad/usuarios/:id/permisos
```

- No se abre una entidad compleja mediante estado temporal de un DataTable.
- Una URL copiada debe cargar correctamente después de refresh.
- Las rutas deben resolver `loading`, `ready`, `not-found`, `forbidden` y `error`.
- Al navegar a una nueva ruta se lleva el scroll al inicio y el foco al H1.
- Los cambios sin guardar activan protección en menú, breadcrumbs, botón atrás, refresh y cierre de pestaña.

6. Tipos de pantallas y wireframes base

Los wireframes se construirán sobre patrones aprobados. Cada nueva pantalla debe seleccionar uno de los patrones base y documentar cualquier desviación.

ID	Patrón	Estructura
WF-01	Listado	PageHeader + resumen opcional + búsqueda/filtro + DataTable + paginación.
WF-02	Formulario de alta/edición	PageHeader + secciones + barra de acciones sticky + guard de cambios.
WF-03	Detalle de entidad	Resumen + estado + pestañas/secciones + timeline + acciones permitidas.
WF-04	Documento transaccional	Cabecera + tercero + líneas + totales + pagos/impuestos + timeline.
WF-05	Matriz	Encabezado + contexto + grupos + columnas compactas + acciones sticky.
WF-06	Asistente	Pasos visibles + validación por paso + resumen + confirmación final.
WF-07	Dashboard	Filtros globales + KPI + alertas + tablas cortas + enlaces a detalle.
WF-08	Reporte	Parámetros + vista previa/estado + exportación + auditoría.
WF-09	Solo consulta	Banner de licencia + controles de escritura

ID	Patrón	Estructura
		bloqueados + lectura/exportación permitida.

6.1 Wireframe base de listado

```
┌──────────────────────────────────────────────────────┐
│ Breadcrumbs                                          │
│ Título                                [Acción primaria] │
│ Descripción breve                                    │
├──────────────────────────────────────────────────────┤
│ [Buscar en esta pantalla.....................] [Estado ▼]  │
├──────────────────────────────────────────────────────┤
│ Tabla sin scroll horizontal                  N registros  │
│ Columnas prioritarias                      Acciones (derecha)│
├──────────────────────────────────────────────────────┤
│ Paginación solo si total > 10                        │
└──────────────────────────────────────────────────────┘
```

6.2 Wireframe base de formulario

```
┌──────────────────────────────────────────────────────┐
│ Breadcrumbs                                          │
│ Título / Estado                    [Volver] [Guardar]    │
├──────────────────────────────────────────────────────┤
│ Sección 1 - Identificación                           │
│ [Campo] [Campo] [Campo]                              │
│ Sección 2 - Datos fiscales                           │
│ [Campo] [Búsqueda asíncrona]                         │
│ Sección 3 - Adjuntos / Auditoría                     │
├──────────────────────────────────────────────────────┤
│ Barra sticky: cambios pendientes   [Cancelar] [Guardar]  │
└──────────────────────────────────────────────────────┘
```

7. Política de modales, diálogos, popovers y tooltips

> **Regla principal**
> KONTROL no utilizará modales para crear o editar entidades, matrices, tablas extensas, configuraciones, documentos, permisos ni procesos con varias secciones. Esos casos deben tener una ruta y pantalla completa.

Componente	Uso permitido	Uso prohibido
Modal/Dialog	Confirmación breve, una decisión, información corta, un dato simple.	Formularios complejos, tabs, DataTables, matrices, scroll extenso.
Drawer	Filtros avanzados móviles, ayuda contextual, detalle secundario no transaccional.	Sustituir formularios principales o esconder pasos críticos.
Popover	Selección o explicación breve vinculada a un control.	Contenido largo, acciones destructivas, formularios complejos.
Tooltip	Nombre de acción o explicación corta en hover/focus.	Información esencial que no exista en otro lugar.

7.1 Requisitos técnicos de diálogos permitidos

- Renderizar mediante portal en `document.body` o `#modal-root`.
- Overlay `position: fixed; inset: 0` que cubra sidebar y topbar.
- Escala de z-index definida; ningún sidebar debe quedar por encima.
- Focus trap, cierre con Escape cuando sea seguro y restauración de foco al elemento invocador.
- Bloqueo de scroll del body y scroll interno vertical solo cuando sea indispensable.
- Sin scroll horizontal.
- En confirmaciones asíncronas, el loader aparece dentro del botón Confirmar; no se abre un segundo modal de carga.
- El diálogo se cierra automáticamente al éxito y la notificación global permanece hasta cierre manual.
- En error, el diálogo permanece abierto y conserva el contexto.

8. Botones, acciones asíncronas, loaders y mensajes

Tipo	Uso	Reglas
Primario	Guardar, emitir, confirmar acción principal.	Máximo uno dominante por contexto; no usar para cancelar.
Secundario	Volver, cancelar, acciones alternativas.	No competir visualmente con el primario.
Destructivo	Eliminar, anular, revocar.	Color de peligro, confirmación y permiso específico.
Icon-only	Acciones de fila y utilidades compactas.	Tooltip, aria-label, foco visible y área táctil mínima.
Link button	Navegación secundaria.	No usar para operaciones que escriben.

8.1 Contrato de toda acción asíncrona

1. Validar permisos, licencia y estado antes de habilitar la acción.
2. Al hacer clic, deshabilitar el botón y evitar doble envío.
3. Mostrar spinner dentro del botón y texto específico: “Guardando…”, “Emitiendo…”, “Eliminando…”.
4. Conservar el ancho del botón para evitar saltos de layout.
5. Enviar clave de idempotencia cuando la operación pueda duplicarse.
6. Esperar confirmación real del servidor; no mostrar éxito anticipado.
7. En éxito, actualizar la caché, cerrar confirmación si aplica y mostrar notificación persistente.
8. En error, normalizar el mensaje, mantener datos y permitir reintento seguro.
9. Registrar request ID para soporte y auditoría.
10. Restaurar el control al terminar salvo que la operación cambie el estado de la página.

8.2 Notificaciones globales

- Deben existir tipos: success, error, warning e info.
- Permanecen hasta cierre manual. No usar auto-dismiss para resultados de operaciones importantes.
- Tienen título, descripción, icono, botón cerrar y, cuando aplique, enlace a detalle.
- No mostrar mensajes técnicos, stack traces, SQL ni respuestas crudas.
- Evitar duplicados mediante clave de notificación.
- No usar únicamente color para comunicar estado.
- Deben ser anunciadas por `aria-live` con nivel adecuado.

Situación	Título recomendado	Detalle
Crear	Registro creado	El registro fue creado correctamente.
Actualizar	Cambios guardados	La información fue actualizada correctamente.
Desactivar	Registro desactivado	El registro ya no podrá utilizarse en nuevas operaciones.
Activar	Registro activado	El registro está disponible para nuevas operaciones.
Eliminar	Registro eliminado	El registro sin relaciones fue eliminado correctamente.
Error genérico	No fue posible completar la operación	Revise la información e intente nuevamente.

9. Formularios, campos y validación

Elemento	Regla obligatoria
Label	Siempre visible; no depender del placeholder.
Requerido	Indicador visual y semántico; no usar solo asterisco sin explicación.
Ayuda	Texto breve bajo el campo o tooltip accesible.
Error	Borde rojo, texto debajo, `aria-invalid`, relación `aria-describedby`.
Foco	Al guardar con errores, enfocar y desplazar al primer campo inválido.
Estado disabled	Visualmente diferenciado y con explicación cuando sea relevante.
Estado read-only	Debe distinguirse de disabled; permite seleccionar/copiar.
Formato	Aplicar al salir o en display, sin impedir edición válida.

9.1 Validación obligatoria

- Validación de cliente para feedback inmediato y validación de backend como autoridad.
- Los esquemas Zod y Pydantic deben derivarse de una misma definición contractual o verificarse en contract tests.
- El mensaje para requerido será exactamente “Campo obligatorio”, salvo regla de dominio más específica.
- No borrar errores del servidor hasta que el usuario cambie el campo relacionado o reintente.
- Los errores de formulario general se muestran en un resumen accesible y en el campo correspondiente.
- No bloquear escritura legítima con máscaras agresivas.

9.2 Tipos de campos

Dato	Control	Regla
RIF, cédula, teléfono, factura, control, cuenta, código, serial, IMEI	Texto	Conservar ceros iniciales y caracteres válidos.
Monto, tasa, porcentaje, cantidad	Decimal	Inicialmente vacío; precisión y escala definidas por dominio.
Fecha	Date	No convertir por zona horaria; formato de pantalla separado del valor.
Fecha/hora	Timestamp con zona	Servidor como autoridad; presentar en zona configurada.
Moneda	Referencia a catálogo	No inferir desde el símbolo ni hardcodear Bs/USD.
Booleano	Switch/checkbox	Label positivo y efecto claro; evitar dobles negaciones.
Catálogo pequeño	Select	Solo cuando el total es reducido y estable.
Catálogo grande	Async combobox	Búsqueda backend, debounce, paginación y cancelación.
Texto largo	Textarea	Contador y límite cuando aplique.
Archivo	File uploader	Tipo, tamaño, progreso, hash, estado y error.

9.3 Números, monedas y porcentajes

- Los inputs numéricos no deben iniciar en `0`; deben aparecer vacíos.
- No convertir automáticamente campo vacío a cero.
- Usar biblioteca decimal o string decimal en API; no usar flotantes para dinero.
- Las etiquetas serán neutrales: “Monto”, “Valor”, “Precio”, “Costo”. La moneda se muestra como indicador dinámico.
- Los porcentajes muestran unidad visual, pero la API recibe la escala definida por contrato.
- El separador decimal y de miles se presenta según locale; el valor interno es independiente.
- Los totales no se recalculan solo en frontend para documentos fiscales; backend devuelve el cálculo definitivo.

9.4 Fechas y tiempo

- La fecha de negocio y la fecha/hora de auditoría son conceptos diferentes.
- El servidor es autoridad de fecha/hora para emisiones, licencias, cierres y auditoría.
- La UI no debe permitir seleccionar fechas fuera de rangos válidos sin explicación.
- Los vencimientos deben mostrar estado: vigente, próximo, vencido.
- Nunca usar el reloj del navegador como única fuente para decisiones de seguridad o licencia.

10. DataTables, búsquedas, filtros y paginación

Regla de tabla

Ningún DataTable debe tener scroll horizontal en las resoluciones soportadas. Se priorizan columnas, se reduce contenido, se usa detalle expandible o se cambia a tarjetas en móvil.

Aspecto	Estándar
Búsqueda	Una búsqueda general por pantalla. Debounce de 300-500 ms y ejecución backend para volúmenes grandes.
Filtro de estado	Cuando exista activar/desactivar: Todos, Activos, Inactivos.
Filtros adicionales	Solo cuando el proceso lo justifique; preferir panel controlado, no una fila de filtros por columna.
Paginación	Mostrar solo si el total supera 10. Opciones de tamaño aprobadas.

Aspecto	Estándar
Orden	Backend para conjuntos grandes; indicador accesible.
Acciones	Última columna, derecha, icon-only, tooltip, aria-label y allowed_actions.
Conteo	Mostrar total de registros y rango visible.
Vacío	Mensaje y acción contextual, no una tabla vacía.
Carga	Skeleton de filas o estado de carga integrado; no bloquear toda la aplicación.

10.1 Prioridad de columnas

- Prioridad 1: identificador, nombre, estado y acciones.
- Prioridad 2: dato operacional esencial.
- Prioridad 3: información complementaria trasladable al detalle.
- En laptop se ocultan columnas de prioridad baja con acceso a detalle.
- En móvil se usa lista de tarjetas; no se comprime la tabla.
- Los textos largos se truncan con tooltip o detalle, sin ampliar indefinidamente la columna.
- Los montos se alinean a la derecha; estados y acciones mantienen anchos compactos.

11. Estados de pantalla y manejo de errores

Estado	Comportamiento
Loading inicial	Skeleton coherente con el contenido; PageHeader puede permanecer visible.
Refetch	Mantener datos previos y mostrar indicador discreto; no vaciar la pantalla.
Empty	Explicar qué falta y ofrecer acción cuando el usuario tenga permiso.
No results	Diferenciar de empty; permitir limpiar búsqueda/filtros.
Error recuperable	Mensaje, acción Reintentar y request ID.
Forbidden	Explicar falta de permiso; no revelar información de la entidad.
Not found	Mensaje claro y enlace a la lista.
Offline/API unavailable	Indicar indisponibilidad; no simular éxito.
Read-only license	Permitir consulta y bloquear escritura con explicación y acceso a licencia.

11.1 Contrato estándar de error API

```
{
  "code": "CUSTOMER_DUPLICATE_TAX_ID",
  "message": "Ya existe un cliente con este RIF.",
  "field_errors": {"tax_id": "RIF ya registrado"},
  "request_id": "req_...",
  "details": null
}
```

- El frontend normaliza 400, 401, 403, 404, 409, 422, 423, 429 y 5xx.
- Los errores 422 se mapean a campos cuando existe ruta de campo válida.
- 409 representa conflicto de negocio o concurrencia; no se muestra como “error desconocido”.
- 423 se utiliza para modo de licencia solo consulta o bloqueo funcional equivalente.
- 401 intenta refresh una sola vez; si falla, cierra sesión de forma segura.
- Nunca mostrar `detail` técnico directamente al usuario.

12. Crear, editar, activar, desactivar, eliminar y anular

Acción	Significado	Regla
Crear	Nuevo registro.	Requiere validación, idempotencia cuando aplique y auditoría.
Editar	Modificar registro permitido.	No aplica a documentos inmutables ni datos fiscales emitidos.
Activar	Habilitar para nuevas operaciones.	Reversible y auditado.
Desactivar	Impedir nuevas operaciones conservando	Reversible; icono X/CircleX; endpoint propio.

Acción	Significado	Regla
	historia.	
Eliminar	Eliminar físicamente un registro sin relaciones.	Solo con `can_delete=true`, validación backend y BD.
Anular	Invalidar un documento emitido según reglas.	No borra; conserva número, estado, motivo y relación.
Reversar	Crear efecto contrario o documento vinculado.	No sobrescribe el evento original.

12.1 Acciones permitidas por registro

```
{
  "allowed_actions": {
    "view": true,
    "edit": true,
    "activate": false,
    "deactivate": true,
    "delete": false,
    "audit": true
  }
}
```

- El frontend no deduce integridad referencial por conteos incompletos.
- El backend y la base de datos vuelven a validar al ejecutar la acción.
- Cuando eliminar no esté permitido, no se muestra el icono; el motivo puede verse en detalle.
- Confirmaciones destructivas deben nombrar la entidad y explicar el efecto.
- No utilizar el mismo icono o endpoint para desactivar y eliminar.

13. Responsive y accesibilidad

Viewport	Comportamiento mínimo
Móvil	Una columna; sidebar tipo drawer; tablas a tarjetas; acciones accesibles; sin controles fuera de pantalla.
Tablet	Una o dos columnas; matrices por secciones; barra de acciones adaptable.
Laptop	Priorizar columnas, ocultar complementarias y conservar acciones.
Desktop	Uso eficiente del ancho; no extender campos cortos innecesariamente.
Zoom 125%-200%	Sin solapamientos, pérdida de acciones ni scroll horizontal de página.

13.1 Accesibilidad WCAG 2.2 AA

- Todo flujo se completa con teclado.
- Foco visible, orden lógico y restauración de foco.
- Un H1 por ruta y jerarquía correcta de encabezados.
- Labels programáticos para campos y botones.
- `aria-live` para notificaciones y estados asíncronos.
- Tooltips visibles en hover y focus.
- No depender solo de color, posición o icono.
- Contraste AA en texto, controles y estados.
- Áreas táctiles mínimas razonables.
- Respetar `prefers-reduced-motion`.
- Las tablas usan encabezados semánticos y nombres de columna.
- Los gráficos tienen resumen textual o tabla equivalente.

14. Estado, caché, API y concurrencia

Tipo de estado	Tecnología/Regla
Servidor	TanStack Query; keys estables por dominio y empresa.

Tipo de estado	Tecnología/Regla
Formulario	React Hook Form; no duplicar en store global.
Navegación	React Router URL/search params.
UI local	useState/useReducer en componente.
Global estable	Context solo para auth, empresa activa, licencia, tema y notificaciones.
Persistencia	Local/session storage solo para preferencias no sensibles.

14.1 Reglas de TanStack Query

- Cada consulta incluye tenant/empresa y parámetros relevantes en su query key.
- No invalidar toda la aplicación después de cada mutación.
- Usar optimistic update solo en acciones reversibles y de bajo riesgo; nunca para emisión fiscal, pagos, movimientos o licencias.
- Cancelar consultas obsoletas en búsquedas.
- No usar datos en caché como autorización.
- Mantener datos previos durante paginación cuando mejore la experiencia.
- Los tiempos de stale y garbage collection se definen por tipo de catálogo.

14.2 Concurrencia

- Entidades editables relevantes deben incluir versión o ETag.
- Al guardar una versión obsoleta, backend responde 409.
- La UI explica que otro usuario modificó el registro y ofrece recargar/comparar; no sobrescribe silenciosamente.
- Las operaciones transaccionales usan bloqueo y reglas de backend, no bloqueos visuales únicamente.
- Los correlativos y números de control se asignan dentro de la transacción final.

15. Seguridad, permisos, licencia y auditoría

Control	Frontend	Backend
Autenticación	Estado de sesión, renovación y cierre seguro.	Validación de tokens, revocación y sesión.
Permisos	Oculta/deshabilita según respuesta.	Autoridad final por acción y alcance.
Multiempresa	Muestra empresa activa.	Filtra y valida cada consulta.
Licencia	Banner y bloqueo visual.	Enforcement en cada operación de escritura.
Auditoría	Muestra timeline y request ID.	Registra evento, usuario, antes/después y origen.
Datos sensibles	Enmascara cuando corresponde.	Control de campo, cifrado y autorización.

15.1 Autenticación

- No almacenar tokens de larga duración en localStorage. Preferir cookies HttpOnly, Secure y SameSite para refresh; access token corto en memoria o cookie según arquitectura aprobada.
- Aplicar CSRF cuando se utilicen cookies para autenticación.
- Cerrar sesión en todas las pestañas mediante canal/broadcast cuando se revoque la sesión.
- No incluir contraseñas iniciales fijas en documentación productiva.
- MFA obligatorio para propietario de KONTROL y perfiles críticos; configurable para administradores del cliente.

15.2 Permisos

- No limitar permisos a CRUD: ver, crear, editar, aprobar, rechazar, activar, desactivar, eliminar, anular, reversar, imprimir, reimprimir, exportar, cerrar, reabrir, configurar y auditar.
- Las rutas deben validar permiso antes de cargar datos sensibles.
- Los botones usan `allowed_actions` del backend.
- Una respuesta 403 siempre se maneja aunque el botón estuviera oculto.
- Los permisos se evalúan con empresa, sucursal, establecimiento, almacén y alcance.

15.3 Licencia anual

- El contexto global de licencia informa FULL_ACCESS, RENEWAL_NOTICE, CRITICAL_RENEWAL_NOTICE o READ_ONLY.
- En READ_ONLY se permiten consulta, reportes y exportación autorizada; se bloquea toda escritura en backend.
- Los botones bloqueados explican el motivo y enlazan a Mi licencia.
- Los trabajos programados e integraciones también deben respetar la licencia.
- El reloj del navegador no decide el estado de la licencia.

16. Reportes, impresión, exportación y archivos

Salida	Tecnología	Regla
PDF empresarial/fiscal	ReportLab backend	Plantilla versionada, datos snapshot, hash y auditoría cuando aplique.
Excel/CSV	Backend	Exportación por streaming, permisos y límites.
Vista previa	Frontend/PDF endpoint	No recrear el cálculo fiscal en el navegador.
Impresión	PDF generado	Reimpresión auditada y distinguible cuando la norma lo requiera.
Adjuntos	Storage persistente	Tipo, tamaño, hash, antivirus opcional, permisos y respaldo.

16.1 Reglas ReportLab

- Todas las plantillas tendrán código y versión.
- Las fuentes se incorporan legalmente y se validan en la imagen Docker.
- Los saltos de página, encabezados, pies, totales y tablas se prueban con contenido extremo.
- Los documentos fiscales no dependen de HTML/CSS del navegador.
- La generación pesada se ejecuta en worker y expone progreso/estado.
- Un fallo de generación no debe marcar un documento como impreso o entregado.
- El archivo final guarda hash, tamaño, tipo, fecha y usuario.

17. Rendimiento y observabilidad

Área	Regla
Bundle	Lazy loading por módulo, análisis de bundle y límites por chunk.
Listados	Paginación backend e índices; no descargar catálogos completos.
Búsqueda	Debounce, cancelación, mínimo de caracteres cuando aplique.
Render	Memoización solo con evidencia; evitar estados globales que rerendericen todo.
Imágenes	Dimensiones, compresión y lazy loading.
API	Timeouts, cancelación, retry solo en operaciones idempotentes.
Logs	JSON estructurado con request ID, usuario, empresa y operación.
Métricas	Latencia, errores, colas, generación de reportes y salud.

17.1 Objetivos mínimos de experiencia

- La interacción visual inicial debe responder inmediatamente con un estado de carga.
- Las búsquedas ordinarias deben percibirse fluidas y no bloquear escritura.
- Los listados nunca deben renderizar miles de filas en el navegador.
- Las operaciones largas muestran estado y permiten continuar trabajando cuando sea seguro.
- No se usan retries automáticos en facturación, pagos, movimientos o acciones destructivas salvo idempotencia comprobada.

18. Pruebas y puertas de calidad

Nivel	Herramienta	Cobertura mínima
Unit frontend	Vitest	Formatos, validadores, permisos, reducers, mappers y utilidades.

Nivel	Herramienta	Cobertura mínima
Componentes	Vitest + Testing Library	Estados, teclado, errores, loaders y accesibilidad.
Contract	OpenAPI/CI	Compatibilidad request/response y errores.
Unit backend	Pytest	Reglas, cálculos, permisos, licencia y servicios.
Integración	Pytest + PostgreSQL	Transacciones, restricciones, migraciones y concurrencia.
E2E	Playwright	Flujos completos, navegadores y roles.
Visual	Playwright/Vitest browser	Pantallas base, responsive, mensajes y modales permitidos.
Accesibilidad	axe + Playwright	WCAG en rutas críticas.

18.1 Matriz obligatoria por pantalla

- Loading inicial.
- Datos cargados.
- Vacío.
- Sin resultados.
- Error API.
- Forbidden.
- Not found.
- Validación de campos.
- Éxito.
- Doble clic.
- Concurrencia.
- Responsive móvil/tablet/laptop/desktop.
- Zoom 125 %, 150 % y 200 %.
- Teclado y lector de pantalla.
- Licencia solo consulta.
- Permisos parciales.

18.2 Puertas de CI

- TypeScript sin errores y modo estricto.
- Lint y formato sin incumplimientos.
- Pruebas unitarias y de contrato aprobadas.
- Migraciones upgrade y downgrade verificadas en entorno de prueba.
- Playwright de flujos críticos aprobado.
- Auditoría de dependencias y secretos.
- Build Docker reproducible.
- Regresión visual revisada cuando cambia el design system.

19. Docker Compose, despliegue y configuración

Cada cliente recibe una instalación independiente en su servidor. El paquete debe poder instalarse, actualizarse, respaldarse y recuperarse sin editar manualmente los contenedores.

Regla	Definición
Imágenes	Versionadas, firmadas cuando sea posible, multi-stage y no root.
Compose	Basado en Compose Specification; archivos separados para producción y desarrollo.
Secretos	Nunca dentro de imágenes, Git o frontend. Variables/secret store con permisos.
Volúmenes	PostgreSQL, storage, licencia y configuración persistentes.
Health checks	Frontend, backend, DB, Redis y worker.
Migraciones	Ejecutadas de forma controlada, con respaldo y rollback documentado.
Seed	Idempotente; no sobrescribe datos del cliente.
Backups	Automáticos, cifrados, verificados y externos al servidor cuando sea

Regla	Definición
	posible.
Logs	Rotación y retención; sin datos sensibles innecesarios.
Actualización	CLI de KONTROL con preflight, backup, pull, migrate, health y rollback.

19.1 Servicios y puertos

- Los puertos públicos se configuran por instalación; PostgreSQL y Redis no deben exponerse a Internet.
- Nginx termina TLS o recibe tráfico desde un proxy seguro del cliente.
- El backend no se expone directamente salvo decisión de infraestructura.
- La documentación Swagger no se expone en producción sin autenticación o configuración explícita.
- Las variables sensibles se validan al inicio y el servicio falla de forma segura si faltan.

20. Estructura de componentes obligatorios

Grupo	Componentes mínimos
Layout	AppShell, Sidebar, Topbar, PageLayout, PageHeader, PageContent, Breadcrumbs, PageActions.
Feedback	PersistentNotification, InlineAlert, ErrorSummary, LoadingButton, ProgressIndicator.
Estados	RouteLoader, EmptyState, NoResultsState, ErrorState, ForbiddenState, NotFoundState.
Formularios	FormField, TextField, NumberField, MoneyField, PercentageField, DateField, AsyncCombobox, FileUploader.
Tablas	DataTable, TableToolbar, GeneralSearch, StatusFilter, Pagination, RowActions, MobileCards.
Acciones	ActivateAction, DeactivateAction, DeleteAction, ConfirmDialog, UnsavedChangesGuard.
Seguridad	PermissionGate, LicenseGate, AllowedActions, SensitiveField.
Documentos	DocumentHeader, LineItemsEditor, TotalsPanel, AuditTimeline, AttachmentsPanel.
Reportes	ReportParameters, ReportStatus, ReportDownload, ExportAction.
Utilidades	Tooltip, Popover, Drawer, Badge, StatusBadge, Skeleton.

20.1 Reglas de componentes compartidos

- Deben tener API documentada, variantes limitadas y ejemplos Storybook o catálogo equivalente.
- Deben incluir estados disabled, loading, error y read-only cuando aplique.
- Deben incluir pruebas de teclado y accesibilidad.
- No pueden contener reglas de negocio específicas de un módulo.
- No se copian y modifican localmente; se amplía la API o se crea componente de dominio.
- Todo cambio que afecte múltiples módulos requiere regresión visual.

21. Definition of Done y checklist por pantalla

Control	Condición de aprobación
Spec	La pantalla y sus estados están definidos en spec.md y wireframe.
Ruta	Tiene URL, breadcrumbs, H1, permisos y error boundary.
Diseño	Usa tokens y componentes aprobados; no CSS ad hoc injustificado.
Feedback	Todas las acciones tienen loader, éxito, error y prevención de doble envío.
Formulario	Campos, tipos, validación, foco y cambios sin guardar completos.
Tabla	Sin scroll horizontal; búsqueda, estado, paginación y acciones estándar.
Responsive	Móvil, tablet, laptop, desktop y zoom verificados.
Accesibilidad	Teclado, labels, foco, aria, contraste y axe aprobados.
Seguridad	Permisos y licencia validados en backend.
Errores	400/401/403/404/409/422/423/5xx manejados.
Auditoría	Evento y request ID disponibles.
Pruebas	Unit, component, integration y E2E según riesgo.

Control	Condición de aprobación
Documentación	Manual de usuario/técnico y catálogo de componentes actualizados.

22. Matriz maestra de reglas

Código	Área	Regla	Severidad
FR-001	Modales	No usar modal para formularios complejos, matrices o tablas.	Bloqueante
FR-002	Rutas	Cada alta, edición, detalle y proceso complejo tiene URL propia.	Bloqueante
FR-003	Botones	Toda acción asíncrona muestra loader dentro del botón.	Bloqueante
FR-004	Doble envío	Botón deshabilitado y operación idempotente cuando aplique.	Bloqueante
FR-005	Éxito	Toda operación exitosa muestra notificación persistente.	Alta
FR-006	Error	Todo error muestra mensaje amigable y conserva contexto.	Bloqueante
FR-007	Confirmación	Loader en Confirmar; no segundo modal de carga.	Alta
FR-008	Notificaciones	No desaparecen automáticamente en operaciones importantes.	Alta
FR-009	Requeridos	Borde rojo, “Campo obligatorio” y foco al primer error.	Bloqueante
FR-010	Numéricos	Inputs inicialmente vacíos, no cero.	Alta
FR-011	Catálogos	Catálogos grandes usan búsqueda asíncrona.	Alta
FR-012	Moneda	Etiquetas neutrales y moneda dinámica.	Bloqueante
FR-013	Tipos	Códigos, RIF, teléfonos, seriales y cuentas son texto.	Bloqueante
FR-014	Tooltips	Acciones icon-only tienen tooltip hover/focus y aria-label.	Alta
FR-015	Clipping	Tooltips/popovers usan portal y collision handling.	Alta
FR-016	Tablas	No scroll horizontal.	Bloqueante
FR-017	Búsqueda	Una búsqueda general por pantalla.	Media
FR-018	Estado	Filtro Todos/Activos/Inactivos cuando aplique.	Alta
FR-019	Paginación	Solo si total > 10.	Media
FR-020	Acciones tabla	Acciones alineadas a la derecha y con iconos estándar.	Alta
FR-021	Eliminar	Solo con can_delete y sin relaciones.	Bloqueante
FR-022	Desactivar	Conserva historia y es reversible.	Bloqueante
FR-023	Allowed actions	Backend retorna acciones permitidas.	Bloqueante
FR-024	Cambios	Guard de cambios sin guardar.	Bloqueante
FR-025	Lista	Conservar búsqueda, página, orden y estado al regresar.	Alta
FR-026	Responsive	Tabla se convierte a tarjetas en móvil.	Alta
FR-027	Accesibilidad	WCAG 2.2 AA y teclado.	Bloqueante
FR-028	Loading	Skeleton o loading integrado; no modal global.	Alta
FR-029	Error boundaries	Rutas y módulos tienen boundaries.	Alta
FR-030	Concurrencia	409 y versión/ETag para edición concurrente.	Bloqueante
FR-031	Licencia	READ_ONLY bloquea escritura en backend y UI.	Bloqueante
FR-032	Permisos	No confiar en ocultar botones.	Bloqueante
FR-033	Auditoría	Request ID y evento para acciones relevantes.	Bloqueante
FR-034	Reportes	PDF definitivo se genera en backend con ReportLab.	Alta
FR-035	OpenAPI	Cliente TypeScript generado; no DTO duplicado manual.	Alta
FR-036	Secretos	No guardar secretos/tokens sensibles en localStorage o frontend.	Bloqueante
FR-037	Dependencias	Versiones fijadas en lockfiles.	Alta
FR-038	Docker	Contenedores no root y health checks.	Alta
FR-039	Pruebas	E2E, accesibilidad y responsive por flujo crítico.	Bloqueante
FR-040	DoD	Ninguna pantalla se cierra sin checklist completo.	Bloqueante

23. Referencias técnicas

El Manual Técnico de INNOVA se utilizó como referencia histórica del stack y de la operación Docker. KONTROL conserva las decisiones que demostraron utilidad y corrige los puntos que generaron retrabajo, especialmente seguridad de tokens, permisos más allá de CRUD, estandarización de componentes, experiencia de formularios y control de despliegue.

- **Manual Técnico - INNOVA v3.0.0:** Referencia interna suministrada por el usuario; arquitectura React, FastAPI, PostgreSQL, Docker Compose, pruebas y estructura modular.
- React: https://react.dev/
- Vite: https://vite.dev/guide/
- Tailwind CSS: https://tailwindcss.com/docs
- TanStack Query: https://tanstack.com/query/latest/docs/framework/react/overview
- React Router: https://reactrouter.com/
- Zod: https://zod.dev/
- FastAPI: https://fastapi.tiangolo.com/
- SQLAlchemy: https://docs.sqlalchemy.org/
- Alembic: https://alembic.sqlalchemy.org/
- PostgreSQL: https://www.postgresql.org/docs/current/
- Docker Compose: https://docs.docker.com/compose/
- ReportLab: https://docs.reportlab.com/reportlab/userguide/ch1_intro/
- Playwright: https://playwright.dev/docs/intro
- Vitest: https://vitest.dev/guide/
- Nginx: https://nginx.org/en/docs/

Aprobación y uso obligatorio

> **Cláusula de aplicación**
>
> Este estándar debe ser incorporado a la Constitución y a todas las Specs de KONTROL. `/speckit.plan`, `/speckit.checklist`, `/speckit.tasks` y `/speckit.analyze` deben comprobar su cumplimiento antes de autorizar cualquier implementación de frontend.

Rol	Aprobación requerida
Producto	Flujo, textos, reglas y criterios de aceptación.
UI/UX	Sistema de diseño, wireframe, responsive y accesibilidad.
Arquitectura	Componentes, estado, API, seguridad y rendimiento.
Backend/Datos	Contratos, integridad, permisos, licencia y auditoría.
QA	Cobertura, pruebas, evidencias y ausencia de regresiones.